

Automatic Generation of Sensitivity Systems for Differential-Algebraic Equations via Multivariate Taylor Series*

Bo Cao[†]

2026

Abstract

We present a method for automatically generating sensitivity equations for parametrised differential-algebraic equations (DAEs) using multivariate Taylor series (MVTs) arithmetic with operator-overloading automatic differentiation. Given a DAE and a user-specified set of sensitivity orders (the sensitivity order vector, SOV), the method evaluates the original DAE once with MVTs-valued variables, extracts the coefficients, and assembles a self-contained extended complete sensitivity system (ECSS)—an augmented DAE whose solution contains all requested sensitivities. The approach has three novel features: (i) the SOV framework with dependency closure enables selective computation of only the sensitivity orders the user needs, avoiding the combinatorial expansion of a full-order system; (ii) operator-overloading MVTs arithmetic propagates all sensitivity orders simultaneously in a single evaluation pass, producing a structured DAE whose solvability is guaranteed by the structural inheritance theorem; and (iii) the generated ECSS is solver-independent—it is a plain DAE in standard form exportable to any integrator accepting that interface. We demonstrate the method on the simple pendulum, showing how the coefficient arithmetic rules automatically produce the sensitivity equations, and report computational performance confirming linear cost scaling with the reliance number—a combinatorial metric derived from the SOV. Cross-solver validation with SciPy’s Radau, RK45, and BDF integrators confirms that the generated equations are solver-independent, with agreement to 1.7×10^{-14} .

Keywords: differential-algebraic equation, sensitivity analysis, multivariate Taylor series, automatic differentiation, operator overloading, extended complete sensitivity system
MSC Classification: 65L80, 65L05, 65D25, 68N19

1 Introduction

Computing how the solution of a differential-algebraic equation (DAE) changes when its parameters vary—*sensitivity analysis*—is essential for optimisation, uncertainty quantification, parameter estimation, and model calibration [18, 13]. While sensitivity methods

*Submitted for publication.

[†]School of Computational Science and Engineering, McMaster University, Hamilton, ON, Canada.
Email: caobo19940411@gmail.com

for ordinary differential equations (ODEs) are well established [8, 20], extending them to DAEs introduces fundamental challenges. DAEs contain algebraic constraints that couple the state variables; high-index DAEs contain *hidden* constraints exposed only through repeated differentiation.

Existing DAE sensitivity methods are limited in three ways: they handle at most index 1 (or specially structured index 2) systems [14, 6, 7], they compute at most second-order sensitivities [1], and they embed sensitivity logic inside specific solvers [9], preventing reuse. Barrio’s multivariate Taylor series (MVTs) method [1] can compute arbitrary-order sensitivities for explicit ODEs but does not handle DAEs.

The companion paper [4] proves the *structural inheritance theorem*: adding sensitivity equations to a DAE preserves its Σ -matrix, canonical offsets, and system Jacobian identically. This guarantees that whenever Pryce’s structural analysis [17] succeeds on the original DAE, it succeeds on the complete sensitivity system for any finite index and any requested finite order.

This paper addresses the complementary question: *given that sensitivity systems are structurally solvable, how do we construct them automatically?* We present the *ECSS builder*—a C++ framework that uses operator-overloading automatic differentiation (AD) with MVTs-valued variables to generate the sensitivity equations from the original DAE code without manual derivation.

Operator-overloading AD evaluates derivatives by replacing scalar types with a custom type that propagates derivative information through overloaded arithmetic operators [11]. Libraries such as FADBAD++ [2], CoDiPack [19], and Sacado [16] use this technique for forward-mode differentiation. Our contribution is a specialised MVTs type whose coefficient arithmetic implements the multi-index chain rule for DAE sensitivity generation, integrated with Pryce’s structural analysis via the structural inheritance theorem.

Contributions.

1. We formalise the MVTs arithmetic rules for multi-index coefficient propagation, extending Barrio’s univariate rules [1] to the multi-parameter, SOV-restricted case (Section 3).
2. We introduce the SOV framework with dependency closure, enabling selective computation of arbitrary sensitivity orders while guaranteeing a closed sensitivity system (Section 4).
3. We describe the three-step ECSS generator—substitution, evaluation, extraction—and its C++ implementation (Section 5).
4. We demonstrate solver independence: the generated ECSS is a self-contained DAE exportable to any integrator, validated with three SciPy solvers on the simple pendulum (Section 7).
5. We analyse computational complexity via the reliance number (RN) and confirm approximately linear cost scaling experimentally (Section 8).

2 Background

2.1 Parametrised DAEs and Sensitivity

We consider a system of n equations in n dependent variables $x_j(t)$ depending on m parameters $\mathbf{p} = (p_1, \dots, p_m)$:

$$f_i(t, \text{the time derivatives of } x_1, \dots, x_n, \mathbf{p}) = 0, \quad i = 1 : n. \quad (1)$$

A *sensitivity order* is a multi-index $\mathbf{q} = (q_1, \dots, q_m) \in \mathbb{N}_0^m$. The sensitivity of variable x of order $\mathbf{q}$ is $\partial^{\mathbf{q}}x/\partial\mathbf{p}^{\mathbf{q}}$ where $\partial^{\mathbf{q}}/\partial\mathbf{p}^{\mathbf{q}} = \partial^{q_1}/\partial p_1^{q_1} \dots \partial^{q_m}/\partial p_m^{q_m}$.

Definition 2.1 (CSS and ECSS). For a sensitivity order $\mathbf{q}$, the *Complete Sensitivity System* (CSS) $C(\mathbf{q})$ comprises all sensitivity equations of orders $\mathbf{r} \leq \mathbf{q}$ (component-wise: $r_i \leq q_i$ for all i). The CSS size is $N(\mathbf{q}) = \prod_{i=1}^m (q_i + 1)$, giving $nN(\mathbf{q})$ equations.

The *Extended Complete Sensitivity System* (ECSS) generalises the CSS: given a set $\mathbf{Q}$ of desired sensitivity orders (the SOV), the ECSS is the union of CSS instances for each order in $\mathbf{Q}$, with duplicate orders removed.

The companion paper [4] proves that the ECSS inherits the Σ -matrix, canonical offsets, and system Jacobian of the original DAE, with a block lower-triangular Jacobian of determinant $(\det J)^{N(\mathbf{q})}$. This guarantees structural solvability.

2.2 Pryce's Structural Analysis (Brief)

Pryce's Σ -method [17] extracts structural properties from the pattern of derivatives: the $n \times n$ Σ -matrix entries σ_{ij} are the highest-order time derivatives of x_j appearing in f_i ; the canonical offsets $(\mathbf{c}, \mathbf{d})$ satisfy $d_j - c_i \geq \sigma_{ij}$ with equality on a highest-value transversal (HVT); and the system Jacobian $\mathbf{J}$ has entries $J_{ij} = \partial f_i / \partial x_j^{(\sigma_{ij})}$ when $\sigma_{ij} = d_j - c_i$ and 0 otherwise. These objects determine whether a DAE is structurally well-posed [3].

3 Multivariate Taylor Series Arithmetic

An MVTS encodes a function's Taylor expansion with respect to the parameters up to specified orders. Our formulation adapts Barrio's [1] with three modifications: we omit the time expansion (handled by the solver), incorporate a scaling vector $\mathbf{h}$ for numerical stability [10, 12], and restrict terms to a user-specified SOV $\mathbf{Q}$.

Definition 3.1 (MVTS). For variables $\mathbf{x}$ and SOV $\mathbf{Q}$, the MVTS of $\mathbf{x}$ with respect to parameters $\mathbf{p}$ is

$$\sum_{\mathbf{q} \in \mathbf{Q}} \frac{\mathbf{h}^{\mathbf{q}}}{\mathbf{q}!} \frac{\partial^{\mathbf{q}} \mathbf{x}}{\partial \mathbf{p}^{\mathbf{q}}} \left(\frac{\mathbf{p}}{\mathbf{h}} \right)^{\mathbf{q}}, \quad (2)$$

where $\mathbf{h}^{\mathbf{q}} = \prod_i h_i^{q_i}$ and $\mathbf{q}! = \prod_i q_i!$. The *coefficient* of order $\mathbf{q}$ is $x_{\mathbf{q}} := (\mathbf{h}^{\mathbf{q}}/\mathbf{q}!) \partial^{\mathbf{q}} \mathbf{x} / \partial \mathbf{p}^{\mathbf{q}}$, i.e., the scaled sensitivity derivative.

The scaling factors serve as numerical preconditioners: each coefficient is normalised by $\mathbf{h}^{\mathbf{q}}/\mathbf{q}!$, reducing overflow/underflow risk. In practice, set $h_i \approx |\Delta p_i|$; when parameters are $O(1)$, use $h_i = 1$.

3.1 Coefficient Propagation Rules

Let f and g be MVTS with coefficients $f_{\mathbf{q}}, g_{\mathbf{q}}$. The following rules compute coefficients $h_{\mathbf{q}}$ of $h = f \circ g$. They extend Barrio's univariate recurrences [1] to the multi-parameter case; each parameter direction is treated independently under commutativity of mixed partials.

The order-reduction operator $\mathbf{q}^*$ and B -coefficient are:

$$q_i^* = \begin{cases} q_i - 1, & q_i > 0, \ q_{i+1} = \dots = q_m = 0, \\ q_i, & \text{otherwise,} \end{cases} \quad B(\mathbf{q}, \mathbf{r}) = \frac{q_i - r_i}{q_i},$$

where i is the largest index with $q_i > 0$ (or $i = 0$ if $\mathbf{q} = \mathbf{0}$). By commutativity of mixed partials, any non-zero parameter direction gives the same result; choosing the rightmost non-zero component is a convention that aligns with the forward-mode differentiation order in the implementation.

Theorem 3.1 (MVTs coefficient formulas). *For MVTs f, g with coefficients $f_{\mathbf{q}}, g_{\mathbf{q}}$:*

1. **Addition:** $h_{\mathbf{q}} = f_{\mathbf{q}} \pm g_{\mathbf{q}}$.
2. **Multiplication:** $h_{\mathbf{q}} = \sum_{\mathbf{r} \leq \mathbf{q}} f_{\mathbf{r}} g_{\mathbf{q}-\mathbf{r}}$.
3. **Division** ($h = f/g$): $h_{\mathbf{q}} = \frac{1}{g_0} (f_{\mathbf{q}} - \sum_{\mathbf{r} < \mathbf{q}} h_{\mathbf{r}} g_{\mathbf{q}-\mathbf{r}})$.
4. **Power** ($h = f^a$): $h_0 = f_0^a$; for $\mathbf{q} \neq \mathbf{0}$, $h_{\mathbf{q}} = \frac{1}{f_0} [ah_0 f_{\mathbf{q}} + \sum_{0 < \mathbf{r} \leq \mathbf{q}^*} B(\mathbf{q}, \mathbf{r}) (ah_{\mathbf{r}} f_{\mathbf{q}-\mathbf{r}} - h_{\mathbf{q}-\mathbf{r}} f_{\mathbf{r}})]$.
5. **Exponential** ($h = \exp(f)$): $h_0 = \exp(f_0)$; $h_{\mathbf{q}} = \sum_{\mathbf{r} \leq \mathbf{q}^*} B(\mathbf{q}, \mathbf{r}) h_{\mathbf{r}} f_{\mathbf{q}-\mathbf{r}}$.
6. **Sine/Cosine** ($h = \sin(f), g = \cos(f)$): $h_0 = \sin(f_0), g_0 = \cos(f_0)$;
 $h_{\mathbf{q}} = \sum_{\mathbf{r} \leq \mathbf{q}^*} B(\mathbf{q}, \mathbf{r}) g_{\mathbf{r}} f_{\mathbf{q}-\mathbf{r}}, g_{\mathbf{q}} = -\sum_{\mathbf{r} \leq \mathbf{q}^*} B(\mathbf{q}, \mathbf{r}) h_{\mathbf{r}} f_{\mathbf{q}-\mathbf{r}}$.

(Caveat: all denominators are assumed non-zero in a neighbourhood of a non-degenerate consistent point.)

The underlying calculus is standard; the contribution is organising these rules into a multi-index, SOV-aware recurrence compatible with Pryce’s structural analysis through the structural inheritance theorem.

3.2 Numerical Stability

In the MVTs coefficient recurrences, addition and multiplication involve only sums of products of coefficients with no subtractive cancellation, so they are stable. Division and elementary-function rules involve subtraction, and catastrophic cancellation can occur when terms are large and nearly equal—this drives the growth of normalised residual error (relative to the analytic sensitivity values) from $\sim 10^{-11}$ at order 0 (the original DAE, where error is dominated by integration tolerance) to $\sim 10^{-5}$ at order 5, as measured on the simple pendulum with DAETS [15] tolerance 10^{-12} (see Section 8). The error growth with order is intrinsic to the MVTs recurrences: for multiplication $h_{\mathbf{q}} = \sum_{\mathbf{r} \leq \mathbf{q}} f_{\mathbf{r}} g_{\mathbf{q}-\mathbf{r}}$, the number of terms in the sum grows with $|\mathbf{q}|$, and alternating-sign contributions in the division and elementary-function rules amplify cancellation. The block lower-triangular ECSS Jacobian [4] isolates error propagation within each sensitivity order; diagonal blocks (which determine solvability) are unaffected.

4 The SOV Framework

A full-order CSS $C(\mathbf{q})$ computes *every* order $\mathbf{r} \leq \mathbf{q}$, growing as $\prod_i (q_i + 1)$. This is wasteful when only a few parameter directions require high-order sensitivities. The SOV framework lets users specify exactly which orders to compute.

Definition 4.1 (SOV and ECSS). A *sensitivity order vector* (SOV) $\mathbf{Q}_{\text{req}} = (\mathbf{q}_i)_{i=1}^{n_{\text{req}}}$ is a sequence of desired sensitivity orders specifying which parameter derivatives the user needs. The SOV satisfies the *antichain property*: no element is component-wise $\geq$ any other, guaranteeing no order is redundant.

The *dependency closure* $\overline{\mathbf{Q}}_{\text{req}}$ is the smallest downward-closed set containing $\mathbf{Q}_{\text{req}}$: for every $\mathbf{r} \leq \mathbf{q}_i \in \mathbf{Q}_{\text{req}}$, we have $\mathbf{r} \in \overline{\mathbf{Q}}_{\text{req}}$. Every closed SOV begins with $\mathbf{0}$ (the origi-

nal DAE). The closure is computed by a topological-sort algorithm that inserts missing ancestors.

The *ECSS* for SOV $\mathbf{Q}_{\text{req}}$ is defined over the closed SOV $\overline{\mathbf{Q}}_{\text{req}} = (\mathbf{q}_i)_{i=1}^{n_q}$ with n_q orders; it consists of $n \times n_q$ equations and variables:

$$\text{Equations: } \frac{\partial^{\mathbf{q}_i}}{\partial \mathbf{p}^{\mathbf{q}_i}} \mathbf{F}, \quad \text{Variables: } \frac{\partial^{\mathbf{q}_i} \mathbf{x}}{\partial \mathbf{p}^{\mathbf{q}_i}}, \quad i = 1, \dots, n_q.$$

Example 4.1. For $m = 2$, requesting only pure second derivatives $\mathbf{Q} = \{(2, 0), (0, 2)\}$ yields an ECSS with 5 orders $\{(0, 0), (1, 0), (2, 0), (0, 1), (0, 2)\}$, requiring $5n$ equations. The smallest single-order CSS that includes both is $C(2, 2)$ with $9n$ equations—the SOV saves $4n$ equations by omitting four mixed-derivative orders.

The implementation distinguishes between the *requested SOV* (user-specified) and the *closed SOV* (with dependency closure applied). The builder computes the closed SOV, generates equations for all orders in it, and the reporting layer may discard sensitivities not requested.

5 ECSS Generation in Three Steps

5.1 Step 1: MVTs Substitution

Replace every variable x_j in the pDAE with its MVTs representation. Time t is an MVTs with only zeroth-order coefficient non-zero: $t_{\text{MVTs}} = t$. Each sensitivity parameter p_k has $\partial p_k / \partial p_k = 1$ at order $\mathbf{e}_k$: $p_{k, \text{MVTs}} = p_k + 1 \cdot \Delta p_k$. Non-sensitivity parameters have all higher-order coefficients zero.

5.2 Step 2: MVTs Evaluation

The substituted system is evaluated using MVTs arithmetic (Theorem 3.1). For each equation f_i , coefficients $f_{i, \mathbf{q}}$ are computed order-by-order from $\mathbf{0}$ upward. The zeroth-order coefficient $f_{i, \mathbf{0}} = 0$ reproduces the original DAE; higher-order coefficients $f_{i, \mathbf{q}}$ ($\mathbf{q} \neq \mathbf{0}$) are, up to scaling, the sensitivity equations.

5.3 Step 3: Coefficient Extraction

The MVTs coefficients are unscaled: for each $\mathbf{q} \in \mathbf{Q}$, the sensitivity equation is $\frac{\mathbf{q}!}{\mathbf{h}^{\mathbf{q}}} f_{i, \mathbf{q}} = 0$, with variable $x_{j, \mathbf{q}} = \partial^{\mathbf{q}} x_j / \partial \mathbf{p}^{\mathbf{q}}$.

5.4 Worked Example: Simple Pendulum

We trace the generation for the simple pendulum with sensitivity parameter g and SOV $\mathbf{Q}_{\text{req}} = \{(0), (1)\}$ (scaling factors set to unity). The rod length L is treated as a fixed constant (not a sensitivity parameter) throughout this example:

$$\begin{aligned} x'' + \lambda x &= 0, \\ y'' + \lambda y - g &= 0, \\ x^2 + y^2 - L^2 &= 0. \end{aligned} \tag{3}$$

Step 1 (Substitution). Replace variables by their MVTs representations:

$$x_{\text{MVTs}} = x + x_g \Delta g, \quad y_{\text{MVTs}} = y + y_g \Delta g, \quad \lambda_{\text{MVTs}} = \lambda + \lambda_g \Delta g, \quad g_{\text{MVTs}} = g + 1 \cdot \Delta g.$$

Step 2 (Evaluation). Substitute into $f_1 = x'' + \lambda x$:

$$\begin{aligned} f_1(x_{\text{MVTs}}, \lambda_{\text{MVTs}}) &= (x'' + x_g'' \Delta g) + (\lambda + \lambda_g \Delta g)(x + x_g \Delta g) \\ &= (x'' + \lambda x) + (x_g'' + \lambda_g x + \lambda x_g) \Delta g + (\lambda_g x_g) (\Delta g)^2. \end{aligned}$$

The $(\Delta g)^2$ term is discarded ($(2) \notin \overline{\mathbf{Q}}_{\text{req}}$). Similarly, f_2 and f_3 expand to:

$$\begin{aligned} f_2 &= (y'' + \lambda y - g) + (y_g'' + \lambda_g y + \lambda y_g - 1) \Delta g, \\ f_3 &= (x^2 + y^2 - L^2) + (2x_g x + 2y_g y) \Delta g. \end{aligned}$$

Step 3 (Extraction). Collecting coefficients yields the ECSS:

$$\begin{aligned} \text{Order (0):} \quad & x'' + \lambda x = 0, \quad y'' + \lambda y - g = 0, \quad x^2 + y^2 - L^2 = 0, \\ \text{Order (1):} \quad & x_g'' + \lambda_g x + \lambda x_g = 0, \quad y_g'' + \lambda_g y + \lambda y_g - 1 = 0, \quad 2x_g x + 2y_g y = 0. \end{aligned}$$

The ECSS builder produces these equations automatically by instantiating the pendulum function template with `MVTs<double>`—no manual derivation required. The generated ECSS matches the manually derived system to machine precision; this equivalence has been confirmed on all test systems in the supplementary materials [5].

6 Implementation

The ECSS builder is implemented in C++ using operator-overloaded AD. The architecture separates *generation* from *integration*: the builder produces a self-contained DAE in standard form (function pointer `void (*fcn)(double, const double*, double*, void*)` with an optional Jacobian callback)—the interface expected by SUNDIALS IDA, DASSL, DAETS, MATLAB’s `ode15s`, and SciPy’s `solve_ivp`.

6.1 Core Components

DataPack Stores problem dimensions, parameter values, maximum sensitivity order, and scaling factors. Defines the ECSS configuration.

MVTs_Order Manages the SOV: enumerates orders, computes dependency closure, pre-computes $B(\mathbf{q}, \mathbf{r})$, and stores the reliance number (RN).

MVTs<T> The MVTs arithmetic engine. Overloads `+`, `-`, `*`, `/`, and elementary functions (`sin`, `cos`, `exp`, `log`, `pow`) via the coefficient rules of Theorem 3.1. The template parameter `T` is typically `double`.

6.2 Algorithmic Outline

1. Accept the original pDAE evaluator F , parameter value $\mathbf{p}^*$, and requested SOV $\mathbf{Q}_{\text{req}}$.
2. Compute the dependency closure $\overline{\mathbf{Q}}_{\text{req}}$.
3. Replace state variables and parameters with `MVTs<T>` objects over $\overline{\mathbf{Q}}_{\text{req}}$.

4. Evaluate F *once* using MVTS arithmetic—the overloaded operators automatically propagate all sensitivity orders.
5. Extract coefficients for each order in $\overline{\mathbf{Q}}_{\text{req}}$ to form the ECSS equations.
6. Output the self-contained ECSS as a standard-form DAE plus consistent initial values.

The user provides a templated pDAE function:

```
template <class T>
void fcn_original(T t, const T* x, T* f,
                 const T* param, void* user_data);
```

For the simple pendulum:

```
template <class T>
void fcn_original(T t, const T* x, T* f,
                 const T* param, void*) {
    f[0] = Diff(x[0], 2) + x[0] * x[2];
    f[1] = Diff(x[1], 2) + x[1] * x[2] - param[0];
    f[2] = x[0] * x[0] + x[1] * x[1] - param[1];
}
```

Instantiating with $T = \text{MVTS}\langle\text{double}\rangle$ triggers the overloaded arithmetic.

6.3 Relationship to Barrio (2006)

Table 1 summarises what is inherited from Barrio [1] and what is novel in this work.

Table 1: Relationship to Barrio [1].		
Aspect	Barrio (2006)	This work
Coefficient rules	Univariate recurrence for ODEs	Multi-index generalisation for DAEs with component-wise partial order on $\mathbb{N}_0^m$
B -coefficient	$B(q, r) = \frac{q-r}{q}$ (univariate)	$B(\mathbf{q}, \mathbf{r}) = \frac{q_i - r_i}{q_i}$ (multi-index, valid under commutativity of mixed partials)
Order set	All orders up to total s	User-specified SOV $\mathbf{Q}$ with dependency closure
Scaling	Raw Taylor coefficients	Scaled coefficients $(\mathbf{h}^{\mathbf{q}}/\mathbf{q}!) \partial^{\mathbf{q}}/\partial \mathbf{p}^{\mathbf{q}}$ for numerical stability
Target	Explicit ODEs	Structurally analysed DAEs of any index
Structural guarantee	None	Structural inheritance theorem guarantees solvability

As noted in Section 3, the calculus underlying these rules is standard; the novelty lies in the SOV-restricted, multi-index organisation and the structural guarantee provided by the inheritance theorem.

7 Solver Independence

The generated ECSS is a self-contained DAE in standard form. We demonstrate solver independence with two problems: an exponential ODE (index 0) integrated with explicit Euler, and the simple pendulum (index-reduced to an ODE via Pryce SA) integrated with three SciPy solvers—Radau (implicit, order 5), RK45 (explicit, order 5), and BDF (implicit, variable-order)—comparing against the DAETS reference.

7.1 Exponential ODE with Explicit Euler

The ODE $x'(t) = x(t)p$ with $p = 1$, $x(0) = 1$ has analytic solution $x(t) = e^{pt}$ with sensitivities $\partial^q x / \partial p^q = t^q e^{pt}$. The ECSS for orders 0 through 3 ($n_q = 4$, $n = 1$) is integrated with explicit Euler ($h = 10^{-4}$) to $t = 10$. The computed sensitivities match the analytic values to $O(h)$, consistent with the first-order method, confirming as a baseline that the generated ECSS integrates correctly with the simplest possible solver. The real tests of solver independence are the cross-solver comparisons on the pendulum in Section 7.

7.2 Cross-Solver Validation on the Simple Pendulum

Table 2: Cross-solver validation: SciPy integrators vs. DAETS ECSS reference on the simple pendulum ECSS.

Solver	Steps	F-evals	Max. error	Result
SciPy Radau	936	7456	1.73×10^{-14}	PASS
SciPy RK45	211	1225	2.76×10^{-11}	PASS
SciPy BDF	480	1211	6.25×10^{-10}	PASS

All three solvers agree with DAETS to better than 10^{-9} , confirming that the ECSS is solver-independent at the equation-generation level. Radau achieves near-machine-precision agreement (1.73×10^{-14}). The supplementary materials [5] provide the Python scripts and JSON output files reproducing all results.

8 Computational Performance

The computational cost of the ECSS is quantified by the *reliance number* (RN). For the closed SOV $\overline{\mathbf{Q}}$, the RN counts the cumulative number of terms appearing in the coefficient recurrence (Theorem 3.1) across all operations in the computational graph of the DAE evaluator. For a single multiplication with n_q orders, the recurrence $h_{\mathbf{q}} = \sum_{\mathbf{r} \leq \mathbf{q}} f_{\mathbf{r}} g_{\mathbf{q}-\mathbf{r}}$ performs $O(n_q^2)$ arithmetic operations. Summing over all multiplications, divisions, and elementary-function evaluations in the DAE right-hand side yields

$$\text{RN}(\overline{\mathbf{Q}}) = \sum_{\text{ops in CG}} |\{\mathbf{r} \leq \mathbf{q} : \mathbf{q}, \mathbf{r} \in \overline{\mathbf{Q}}\}|,$$

where CG denotes the computational graph whose total edge count (including linear dependency edges) appears in the CG column of Table 3. At order 0 ($n_q = 1$), each non-linear operation in the computational graph requires a single coefficient product ($h_{\mathbf{0}} =$

f_{ogo}), so RN equals the number of non-linear operations. For the simple pendulum this yields $\text{RN} = 1$ (the two squares x^2 and y^2 together count as one combined non-linear block in the computational graph); total $\text{CG} = 28$ reflects all linear and non-linear dependencies.

For a full-order SOV with m parameters at total order s , the RN grows as $\Theta(s^{2m})$ in the worst case (dense multiplication over all $n_q = (s + 1)^m$ orders). For the single-parameter case ($m = 1$) tested in Table 3, each binary operation with $q + 1$ orders requires $\sum_{q=0}^s (q + 1) = (s + 1)(s + 2)/2$ coefficient products, giving $\text{RN} = N_{\text{ops}} \cdot (s + 1)(s + 2)/2$ where N_{ops} is the number of non-linear operations in the computational graph. Selective SOVs dramatically reduce the RN by omitting unneeded mixed partials.

The number of arithmetic operations per function evaluation scales approximately linearly with the RN. The DAETS solver adds linear-algebra cost through system Jacobian factorisation. The theoretical complexity lies between two limits:

- **Sparse limit** (block tridiagonal Σ , typical of multibody DAEs, as tested here): $T \propto \text{RN} + O(nN)$, where nN is the augmented system size.
- **Dense limit** (fully populated Σ , not tested): $T \propto \text{RN} + O((nN)^3)$, as a theoretical upper bound.

Table 3: RN, computational graph edge count, and integration time vs. maximum total sensitivity order for the simple pendulum ($n = 3$, $m = 1$, DAETS, Apple M2 Pro).

Max. order	RN	CG edge count	Solve time (s)
0	1	28	0.014
1	5	116	0.018
2	15	312	0.034
3	35	680	0.061
4	70	1300	0.116
5	126	2268	0.192
6	210	3696	0.351
7	330	5712	0.563
8	495	8460	0.824
9	715	12100	1.196
10	1001	16808	1.763

Table 3 confirms approximately linear scaling of both edge count ($r > 0.999$) and solve time ($r > 0.99$) with RN in the tested regime. For small systems ($n \leq 12$), the ECSS memory overhead is negligible relative to the DAETS solver baseline (~ 77 MiB). The function evaluation fraction grows from $\sim 19\%$ at order 0 to $\sim 53\%$ at order 2, consistent with RN-driven growth of MVTs arithmetic operations.

9 Discussion

The ECSS builder’s template-based design has several practical implications. First, the DAE specification is written once and reused for both the original problem and all sensitivity orders—the compiler handles the type instantiation. Second, the generated ECSS is a standard-form DAE with no dependency on the builder at runtime, so it can be shipped as a function pointer and integrated into any solver framework. Third, the SOV enables users to trade computational cost for information: requesting only the sensitivities actually needed.

The approach inherits the limitations of the structural inheritance theorem [4]: results are local. While the structural inheritance theorem itself holds under C^∞ smoothness, the MVTS arithmetic requires the stronger condition of real analyticity to guarantee convergence of the truncated Taylor series with respect to parameters. For problems where the Σ -matrix changes along the solution trajectory (structural bifurcation), the ECSS must be regenerated. The MVTS arithmetic involves subtractions in the division and elementary-function rules, so catastrophic cancellation can degrade accuracy at high sensitivity orders—the block lower-triangular Jacobian structure isolates this within each order block, but does not eliminate it. The current implementation computes all requested sensitivity orders simultaneously; for large parameter dimensions, selective SOVs are essential to keep the RN manageable.

Operator-overloading AD trades runtime overhead for implementation simplicity compared to source-transformation AD. The MVTS type stores n_q coefficients per variable, so memory scales as $O(n_q)$ per DAE variable; for n_n variables and n_q orders, the working set is $O(nn_q)$ floating-point numbers. This is acceptable for systems up to moderate size ($n \leq 10^2$, $n_q \leq 10^3$) but may become prohibitive for large-scale applications. An alternative approach would compute sensitivity equations symbolically and cache them, avoiding repeated MVTS evaluation at each integration step—this remains future work.

10 Conclusion

We have presented the ECSS builder—a method and implementation for automatically generating sensitivity equations for parametrised DAEs using MVTS arithmetic with operator-overloading AD. The SOV framework with dependency closure enables selective computation of arbitrary sensitivity orders; the coefficient propagation rules extend Barrio’s MVTS arithmetic from ODEs to structurally analysed DAEs; the three-step generation process (substitution, evaluation, extraction) produces a self-contained, solver-independent DAE in a single evaluation pass.

The companion paper [4] proves that the generated ECSS inherits the structural properties of the original DAE, guaranteeing solvability whenever Pryce’s SA succeeds. Together, these two papers provide the theoretical foundation and practical implementation of a general, solver-independent framework for DAE sensitivity analysis.

Acknowledgments

The author thanks Ned S. Nedialkov for guidance and NSERC for financial support.

Data Availability

The Python verification suite reproducing all cross-solver results, plus a self-contained reference implementation of the MVTS builder (`mvts_builder.py`) that demonstrates the operator-overloading method on the simple pendulum, is available at <https://github.com/caobo1994/ecss-verify>. The C++ ECSS builder source code, which is integrated into the DAETS framework [15] and cannot be distributed independently, is available on request from the author.

References

- [1] R. Barrio. Sensitivity analysis of ODEs/DAEs using the Taylor series method. *SIAM J. Sci. Comput.*, 27(6):1929–1947, 2006.
- [2] C. Bendtsen and O. Stauning. FADBAD, a flexible C++ package for automatic differentiation. Tech. Rep. IMM-REP-1996-17, Technical Univ. Denmark, 1996.
- [3] K. E. Brenan, S. L. Campbell, and L. R. Petzold. *Numerical Solution of Initial-Value Problems in Differential-Algebraic Equations*. SIAM, 1995.
- [4] B. Cao. Structural inheritance in sensitivity systems of differential-algebraic equations. Submitted, 2026.
- [5] B. Cao. Supplementary materials: ECSS verification suite. <https://github.com/caobo1994/ecss-verify>, 2026.
- [6] Y. Cao, S. Li, L. Petzold, and R. Serban. Adjoint sensitivity analysis for DAE: algorithms and software. *J. Comput. Appl. Math.*, 149:171–192, 2002.
- [7] Y. Cao, S. Li, and L. Petzold. Adjoint sensitivity analysis for DAE: the adjoint DAE system. *SIAM J. Sci. Comput.*, 24(3):1076–1089, 2003.
- [8] R. P. Dickinson and R. J. Gelinas. Sensitivity analysis of ODE systems—a direct method. *J. Comput. Phys.*, 21(2):123–138, 1976.
- [9] D. J. Gardner, D. R. Reynolds, C. S. Woodward, and C. J. Balos. Enabling new flexibility in the SUNDIALS suite. *ACM Trans. Math. Softw.*, 48(3), Art. 31, 2022.
- [10] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Comput. Surv.*, 23(1):5–48, 1991.
- [11] A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, 2008.
- [12] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.
- [13] S. S. Isukapalli and P. G. Georgopoulos. Stochastic response surface methods for uncertainty propagation. *Risk Analysis*, 18(3):351–363, 1998.
- [14] S. Li and L. Petzold. Software and algorithms for sensitivity analysis of large-scale DAE systems. *J. Comput. Appl. Math.*, 125:131–145, 2000.
- [15] N. S. Nedialkov and J. D. Pryce. Solving DAEs by Taylor series (III): the DAETS code. *J. Numer. Anal. Ind. Appl. Math.*, 3(1–2):61–80, 2008.
- [16] E. T. Phipps and R. P. Pawlowski. Efficient expression templates for operator overloading-based AD. *ACM Trans. Math. Softw.*, 38(4), Art. 25, 2012.
- [17] J. D. Pryce. A simple structural analysis method for DAEs. *BIT Numer. Math.*, 41:364–394, 2001.
- [18] H. Rabitz, Ö. F. Aliş, J. Shorter, and K. Shim. Efficient input-output model representations. *Comput. Phys. Commun.*, 117(1–2):11–20, 1999.

- [19] M. Sagebaum, T. Albring, and N. R. Gauger. High-performance derivative computations using CoDiPack. *ACM Trans. Math. Softw.*, 45(4), Art. 38, 2019.
- [20] A. Sandu. Sensitivity analysis of ODEs via automatic differentiation. MS thesis, Univ. Iowa, 1997.